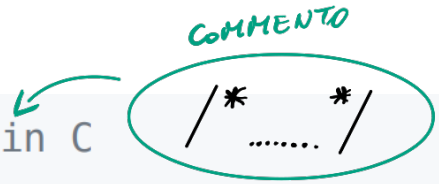
PROGRAMMAZIONE IN C

ESAME ORALE: 12/01/2024

#include <stdio.h>




```c
//Il mio primo programma in C

#include <stdio.h>

int main()
{
  printf("Hello world");
  return 0;
}
```

- E' una direttiva al preprocessore
- Le linee che iniziano con # vengono elaborate prima della compilazione del programma
- Nel programma verrà incluso il contenuto del file stdio.h
- Stdio.h (standard input-output header) è un file di intestazione che definisce gli stream (flussi) di input e output.

Int main()

- Il main è la funzione principale
- Tutte le funzioni vengono richiamate usando il loro nome e tra () vengono indicati i parametri
- Int indica che la funzione restituisce un intero e le parentesi {} definiscono il corpo della funzione

Printf("Hello world\n")

- La printf è una funzione che scrive sullo standard output (monitor)
- Tra "…." si può inserire del testo
- \n indica l'inserimento di una nuova linea
- Ogni istruzione deve terminare con un ;

Return 0

- Indica che il programma è terminato con successo

Dichiarazione di variabili

- E' il nome (etichetta) di un'area di memoria che il nostro programma potrà successivamente utilizzare
- Dichiarazione delle variabili: definizione delle aree di memoria che saranno utilizzate nel programma
- Assegnazione di un nome (identificatore) e attribuzione del tipo
- Il tipo definisce le caratteristiche che regolano l'uso di una variabile

```
tipo nomevariabile;
tipo nomevariabile1, nomevariabile2;
```

Int - char – float – const - #define

```
const float PiGreco = 3.14;
const int N = 100;
```

```
#define N 100

//direttiva al preprocessore
//non e' scope-controlled
```

Vantaggi e svantaggi delle dichiarazioni costanti

- Le costanti sono valori fissi che mantengono lo stesso valore durante l'esecuzione del programma. Contrariamente alle variabili, il valore di una costante non può essere modificato dopo essere stato assegnato. Vantaggi chiave nell'utilizzo delle costanti:
  - Leggibilità: L'uso di costanti con nomi significativi rende il codice più chiaro e leggibile
  - Manutenibilità: Le costanti consentono di apportare facilmente modifiche a valori comuni in un programma senza dover cercare e modificare manualmente ogni occorrenza
  - Sicurezza: L'uso di costanti impedisce la modifica accidentale dei valori durante l'esecuzione del programma, riducendo gli errori potenziali1.
  - Portabilità: Le costanti possono essere utilizzate per rappresentare valori che potrebbero variare tra diversi sistemi o ambienti senza dover modificare il codice sorgente
- Tuttavia, l'utilizzo eccessivo di costanti può portare a svantaggi come la riduzione della flessibilità del codice e la creazione di un codice più prolisso. Inoltre, le costanti definite con #define sono sostituzioni di testo che vengono effettuate dal preprocessore prima della compilazione. Ciò può portare a problemi di debug e di manutenzione del codice

- Int e Char sono due tipi utilizzati dal linguaggio C per la rappresentazione di valori interi
- Int sono assegnati 32 bit
- Char sono assegnati 8 bit
- La funzione float() restituisce un numero a virgola da un numero o da una stringa. float() prende un singolo parametro ovvero il numero o la stringa che deve essere convertito in numero a virgola mobile, se è una stringa la stringa dovrebbe contenere punti decimali
- Const parola chiave specifica che il valore di una variabile è costante e indica al compilatore di impedire al programmatore di modificarlo
- #define N 10 inserimento massimo dei valori 10 (da 0 a 9)

Istruzione di assegnamento

- Viene utilizzato il simbolo =
- Permette di assegnare ad una variabile un valore o il risultato di una espressione (o il valore restituito da una funzione)
- Identificatore = valore
- ```
  a=0;
  c='a';
  x=x+1; //Come si interpreta?
  ```
  Assegna il valore **0** alla variabile **a**, il carattere **'a'** alla variabile **c** e incrementa il valore della variabile **x** di **1**

- L'operatore == in C è un operatore di confronto che confronta due espressioni e restituisce 1 se sono uguali, 0 altrimenti

- + addizione
- - sottrazione
- * moltiplicazione
- / divisione
- % modulo (resto della divisione)

- Standard output: printf() Stampa su video
- Standard input: scanf() Inserimento di valori (da tastiera)

- %c formato di stampa: carattere
- %d formato di stampa: intero sistema decimale

Istruzione scanf

- Inserimento di valori (da tastiera)

```
scanf("%d", &base);
```
```

- Cosa indica il simbolo &?
    - Il simbolo & in C è l'operatore di indirizzo. Esso restituisce l'indirizzo di memoria di una variabile
- Perché si usa il simbolo &?
    - Il simbolo & viene utilizzato per passare un parametro per riferimento ad una funzione. Ciò significa che la funzione si aspetta un puntatore
- Cosa succede se non uso il simbolo &?
    - Se non usi il simbolo &, la funzione riceverà una copia del valore della variabile invece del suo indirizzo di memoria ma il programma si comparta in modo anomalo

Istruzioni composte

Unione di operatori aritmetici e assegnazione:

`operatore_aritmetico=` (" `+=` ")

```
count += 10;
```

somma l'espressione a destra con l'espressione a sinistra e assegna alla variabile a sinistra

```
count = count + 10;
```

Tipi di dato

- Il tipo di dato specifica un insieme di valori e un insieme di possibili operazioni che possono essere applicate ad una variabile
- Ogni tipo ha una propria rappresentazione (codifica) in memoria attraverso un'opportuna sequenza di bit
- I linguaggi di programmazione permettono un certo livello di astrazione (possiamo ignorare come l'informazione viene rappresentata in memoria)
- Ogni variabile deve avere un tipo (assegnato dalla dichiarazione)

Classificazione dei tipi di dato

- Tipi semplici: per rappresentare informazioni semplici
- Tipi strutturati: per rappresentare informazioni costituite da diverse componenti

Tipi semplici

- int (interi) La dimensione di una variabile di tipo int (generalmente) è di 4 byte
- float (reali)
- double (reali con precisione doppia)
- char (caratteri)

Ogni tipo è associato ad un intervallo di valori che è legato alla quantità di memoria allocata (dipende dalla macchina)

Tipo int - interi

```
int n = sizeof(int);
```

- La dimensione di una variabile di tipo int (generalmente) è di 4 byte
- Il valore restituito dall'operatore sizeof applicato al tipo di dato int alla variabile n. In altre parole, la variabile n conterrà il numero di byte necessari per rappresentare un intero
- printf("%ld" ,...) -> long
- printf("%hd" ,...) -> short

Tipo float

```
float n;
```

- Numeri con parte frazionaria: virgola mobile (floating point)
- La dimensione di una variabile di tipo float (generalmente) è di 4 byte
- printf("%f" ,...) -> float
- printf("%2f" ,...) -> .00float
- printf("%.3f" ,...) -> .000 float

Tipo double

- Permette di avere una maggiore precisione
- La dimensione di una variabile di tipo double (generalmente) è di 8 byte
- Riduce l'effetto degli errori di approssimazione (arrotondamento)

Tipo char

- I caratteri possono assumere valori alfanumerici
- La dimensione di una variabile di tipo char (generalmente) è di 1 byte

```
char n;
n = 'A';
```

Operatori relazionali

- == uguaglianza
- != diversità
- < minore di
- > maggiore di
- <= minore o uguale a
- >= maggiore o uguale a

Operatori logici

- Permettono di concatenare più espressioni logiche
- ! not
- && and
- || or

Istruzione if (condizione se)

```
if (condizione) {istruzioni} else {istruzioni}
```

- Consente di eseguire un'istruzione al verificarsi di una condizione
- Oppure di eseguire in alternativa un'altra istruzione

```
if (condizione) {istruzioni} else {istruzioni}
```

- Se la condizione è seguita da una sola istruzione le parentesi graffe possono essere omesse

```
if(x==0) printf("x vale 0");
```

- controlla se il resto della divisione di n per 2 è uguale a 0. Se la condizione è vera, allora il codice all'interno delle parentesi graffe viene eseguito. Altrimenti, il codice viene ignorato e il programma continua con l'esecuzione del codice successivo

```
if(n%2 == 0)
        printf("\n%d e' pari\n",n);
else
        printf("\n%d e' dispari\n",n);

return 0;
```

switch

```
switch (giudizio)
{
  case 1:
      printf("Hai assegnato 1!\n");
      printf("Ci dispiace...\n");
      break;
  case 2:
      printf("Hai assegnato 2!\n");
      printf("Grazie!\n");
      break;
  case 3:
      printf("Hai assegnato 3!\n");
      printf("Grazie mille!.\n");
      break;
  default:
      printf("Giudizio non valido!\n");
}
return 0;
```

- Il costrutto if-else talvolta risulta innaturale
- Lo switch permette di selezionare un'istruzione (tra un insieme di istruzioni possibili) in base al valore assunto una variabile (o da un'espressione)
- Confronto tra risultato espressione (int o char) e un insime di valori costanti (int o char)

Iterazione

```
#include <stdio.h>
int main()
{
  int i,n,somma=0;

  for(i=1;i<=3;i=i+1)
  {
    scanf("%d",&n);
    somma=somma+n;
  }
  printf("%d",somma);
```

- Permette di sviluppare programmi con processi ripetitivi (cicli), evitando di scrivere più volta la/e stessa/e istruzione/i.

```
for(i=1;i<=3;i=i+1)
```

Questo ciclo viene eseguito 3 volte, poiché la condizione i<=3 è soddisfatta per i valori di i da 1 a 3. Ad ogni iterazione del ciclo, la variabile i viene incrementata di 1. Quindi, alla fine del ciclo, i avrà il valore 4

Il ciclo For

```
for(i=1;i<=3;i=i+1)
{
  printf("\nInserisci numero %d: ",i);
  scanf("%d",&n);
  somma=somma+n;
```

- For (inizializzazione; condizione; incremento/decremento) istruzione;
- Inizializzazione: viene eseguita solo la prima volta
- Condizione: condizione necessaria affinché il ciclo continui
- Incremento/decremento: ultima espressione del ciclo

Attenzione ai cicli infiniti! for(i=1;i>=1;i=i+1)

Operatore: incremento/decremento

`i++;` equivale a `i=i+1;`

`i--;` equivale a `i=i-1;`

Attenzione:

`b=2; a=++b;` b=3 e a=3

`b=2; a=b++;` b=3 e a=2

somma+=i equivale a somma=somma+i

```
for(i=1;i<=n;++i)
```

→ Questo ciclo viene eseguito n volte, poiché la condizione i<=n è soddisfatta per i valori di i da 1 a n. Ad ogni iterazione del ciclo, la variabile i viene incrementata di 1. Quindi, alla fine del ciclo, i avrà il valore n+1

```c
#include <stdio.h>
int main()
{
  int n,i,voto,somma=0,soglia,sopra=0;

  printf("Quanti voti?");
  scanf("%d",&n);
  printf("Che soglia?");
  scanf("%d",&soglia);
  for(i=1;i<=n;++i)
  {
    printf("Inserisci voto %d: ",i);
    scanf("%d",&voto);
    somma+=voto;
    if(voto>soglia) ++sopra;
  }
  printf("\nLa media e': %.1f\n",(float)somma/n);
  printf("\n%d voti sopra %d", sopra,soglia);
}
```

While

```c
while(i<=3)
{
  printf("\nInserisci numero %d: ",i);
  scanf("%d",&n);
  somma=somma+n;
  i++;
}
```

```
while(condizione) {istruzioni;}
```

- Il ciclo while impone la ripetizione del blocco di istruzioni fino a quando la condizione non diventa FALSE
  - Valutazione della condizione
  - Se condizione è FALSE -> non viene eseguito il corpo del ciclo
  - Si valuta nuovamente la condizione e così via…

L'inizializzazione della variabile che controlla il ciclo deve essere fatta prima del while. L'incremento sarà eseguito come ultima istruzione del corpo del while

```c
#include <stdio.h>
int main() {
        float altezza,peso,bmi;
        char sel='Y';
        while(sel!='N') {
                printf("Inserisci altezza in m:\n");
                scanf("%f",&altezza);
                printf("Inserisci peso in Kg:\n");
                scanf("%f",&peso);
                bmi=peso/(altezza*altezza);
                if(bmi<=20)
                        printf("BMI: %.2f; sottopeso\n",bmi);
                        else if (bmi>20 && bmi<=30)
                                printf("BMI: %.2f; normopeso\n",bmi);
                                else printf("BMI: %.2f; sovrappeso\n",bmi);
                printf("N per uscire, Y per continuare: \n");
                scanf(" %c",&sel);
                while(sel!='N' && sel!='Y')             {
                        printf("Scelta non valida \n");
                        printf("N per uscire, Y per continuare: \n");
                        scanf(" %c",&sel);
                }}}
```

- Char sel='Y' la variabile sel è di tipo char, può contenere un singolo carattere
- bmi>20 && bmi<=30 controlla se il valore di bmi si trova nell'intervallo tra 20 e 30
- sel!='N' && sel!='Y') questa espressione controlla se il valore di sel è diverso da 'N' e 'Y'
- scanf(" %c",&sel) indica che la funzione scanf() leggerà un singolo carattere dalla tastiera e lo memorizzerà nella variabile sel. L'uso dello spazio vuoto prima del carattere %c consente di ignorare eventuali spazi o caratteri di nuova riga rimasti nel input

Do While

- Le istruzioni for e while verificano la condizione prima che il ciclo venga eseguito
- Il corpo del ciclo potrebbe non essere eseguito mai!
- Se vogliamo verificare la condizione alla fine del ciclo possiamo usare il do-while

```c
do istruzione; while(espressione);
```

```c
#include <stdio.h>
int main()
{
  int n=0;

  do printf("%d ",n++);
  while(n<=9);
}
```

Array

- È un tipo di struttura dati che memorizza una raccolta sequenziale di elementi dello stesso tipo
- Tipo derivato: deve far riferimento ad un tipo base
- Quando si usa un array. Quando abbiamo necessità di trattare un insieme omogeneo di dati
- È quindi una variabile strutturata, i cui elementi sono tutti dello stesso tipo (tipo del vettore) **Dichiarazione di un array: Indicare nome, tipo e lunghezza**

```
int vett[10];
```

- Con la dichiarazione riserviamo lo spazio di memoria sufficiente per il nostro array
- In questo esempio, evitiamo di dover dichiarare 10 diverse variabili
- Accedere agli elementi di un array:
    - Tutti gli elementi di un array hanno lo stesso nome: nome_array
    - Ogni elemento è identificato da un indice: nome_array[indice]
    - Esempio: vett[2] //permette di accedere all'elemento con indice 2

- Il primo elemento di un array ha indice `0` ❗
- L'ultimo elemento di un array ha indice `N-1`, dove `N` indica la dimensione dell'array

```
[0] [1] [2] ... [N-1]
```

Tentare di accedere con un indice diverso (fuori dall'intervallo) può non generare un errore 😕

Assegnamento

```
int vett[10], num;

vett[0]=5;

num=vett[0]; //assegna 5 a num
```

Permettono di sviluppare programmi concisi ed efficienti

- Esempio:

```
int voti[10],somma=0,i;

...

for(i=0; i<10; ++i) // <- i parte da 0
{
        printf("Inserisci voto: ");
        scanf("%d",&voti[i]); // <- conserviamo i voti
        somma += voti[i];
}
```

Considerazione importante!

- Int voti[10]; stiamo dicendo che la lunghezza dell'array sarà 10!
- Se volessimo inserire 20 elementi?
    - Modifica noiosa e rischiosa!
    - Cambiare ogni occorrenza!
    - Possibili diversi significati del numero 10!

Utilizzo della direttiva #define (si usa un nome simbolico)

```
#define NVOTI 10
int voti[NVOTI];
```

```
#include <stdio.h>
#define NVOTI 10

int main()
{
  int i,n,voti[NVOTI],somma=0;
  float media;
  printf("Quanti voti vuoi inserire? ");
  scanf("%d",&n);
  for(i=0;i<n;i++)  {
    printf("\nInserisci voto %d: ",i+1);
    scanf("%d",&voti[i]);
    somma+=voti[i];
  }
  media=(float)somma/n;
  printf("\nLa media vale: %f\n",media);
}
```

Note

- Il C non effettua alcun controllo sui limiti degli array
- In altre parole, è possibile inizializzare un array di dimensione N con più di N valori senza avere alcun messaggio di errore in compilazione
- È compito del programmatore garantire che tutti gli array siano abbastanza grandi da contenere ciò per cui sono stati creati

Matrici

- Vettori: array monodimensionali
- Matrici: array bidimensionali
  - Il C permette di definire array di qualsiasi dimensione
- Per accedere ad un elemento di una matrice: M[i][j]
  - Il primo indice identifica la riga, il secondo la colonna
- Dichiarazione: int M[NR][NC]

```c
#include <stdio.h>
#define DIM 3
int main()
{
    int m[DIM][DIM],i,j;

    printf("Inserisci elementi nella matrice:\n");
    for(i=0;i<DIM;i++) {
        for(j=0;j<DIM;j++) {
            printf("elemento - [%d],[%d] : ",i,j);
            scanf("%d",&m[i][j]);
        }
    }
    printf("\nLa matrice e': \n");
    for(i=0;i<DIM;i++) {
        printf("\n");
        for(j=0;j<DIM;j++)
            printf("%d\t",m[i][j]);
    }
}
```

Puntatori

- Dichiarazione di variabile: int n;
  - nome
  - locazione di memoria
  - indirizzo della locazione di memoria
- L'operatore & (vedi utilizzo scanf) restituisce l'indirizzo di memoria di una variabile
- I puntatori sono delle variabili alle quali può essere assegnato un indirizzo di memoria

Perché usare i puntatori e non le variabili?

Le variabili e i puntatori sono entrambi utilizzati per memorizzare i dati ma hanno usi e comportamenti diversi:

Variabile

- Una variabile è un nome dato a una posizione di memoria in cui si memorizzano i dati.
- Il valore di una variabile è il dato effettivo che è stato memorizzato in quella posizione di memoria.
- Quando si modifica il valore di una variabile, si sta cambiando il dato in quella posizione di memoria.

Puntatore

- Un puntatore è una variabile speciale che memorizza l'indirizzo di un'altra variabile.
- Il valore di un puntatore è un indirizzo di memoria.
- Quando si modifica il valore di un puntatore, si sta cambiando l'indirizzo di memoria a cui punta, non il dato all'indirizzo di memoria.

In sintesi, una variabile contiene un valore, mentre un puntatore contiene un indirizzo di memoria. Un puntatore può essere usato per accedere e modificare direttamente il valore della variabile a cui punta. Questo può essere utile per operazioni come la manipolazione diretta della memoria e il passaggio di grandi strutture di dati a funzioni senza dover copiare l'intera struttura. Tuttavia, l'uso dei puntatori può essere più complesso e può portare a errori se non gestito correttamente

Puntatori: dichiarazione

```
int *int_pointer;
```

Occorre specificare il tipo base (in questo caso int) e il nome della variabile (in questo caso int_pointer) preceduto dal simbolo *

```
int n=10;
int *p;

p=&n; //non assegna a p il valore di n!
```

- In questo caso la variabile puntatore ad intero p è inizializzata con l'indirizzo di n
- L'operatore * (detto di indirezione) applicato ad una variabile puntatore permette di restituire il valore contenuto nella variabile puntata

```
#include <stdio.h>

int main()
{
  int n=10,m;
  int *p,*q;

  p=&n;
  m=*p;
  n=20;
  printf("%d %d %d",n,*p,m); //stampa: 20 20 10

  *p=30;
  printf("%d %d %d",n,*p,m); //stampa: 30 30 10

  q=p;
  printf("%d %d %d",n,*p,*q); //stampa 30 30 30
}
```

Puntatori e Array

Il nome di un array rappresenta un puntatore costante al suo primo elemento: punta sempre al primo elemento e non è possibile assegnare l'indirizzo di un'altra cella di memoria

```
int vett[N];
int *vett_ptr;

vett_ptr = &vett[0];
```

equivale a

```
vett_ptr = vett; //non compare &
```

E' quindi possibile accedere agli elementi di un array utilizzando un puntatore.

```
#include <stdio.h>
int main() {

  int i,vett[3]={1,2,3};
  int *vett_ptr;

  for(i=0;i<3;i++)
    printf("%d ",vett[i]); //1 2 3

  vett_ptr=vett; //vett_ptr punta dove punta vett
  *(vett_ptr+1)=10;

  for(i=0;i<3;i++)
    printf("%d ",vett[i]); //stampa 1 10 3
}
```

Che differenza c'è tra:

```
  for(i=0;i<3;i++)
    printf("%d ",*(vett_ptr++));
}
```

e

```
  for(i=0;i<3;i++)
    printf("%d ",*(vett_ptr+i));
}
```

La differenza tra vett_ptr++ e vett_ptr+i è che il primo incrementa il puntatore di un elemento, mentre il secondo incrementa il puntatore di i elementi

Aritmetica dei puntatori

- Incremento: permette di passare ad un indirizzo successivo p++ , p+1 , p+i
- Decremento: permette di passare ad un indirizzo precedente p-- , p-1 , p-i
- Lo spostamento dipende dal tipo base!

Puntatori e Array: ATTENZIONE

- Domanda___Differenza tra vett_ptr++ e ++vett_ptr??
- Risposta____ La differenza tra vett_ptr++ e ++vett_ptr è che il primo incrementa il puntatore di un elemento e restituisce il valore originale del puntatore, mentre il secondo incrementa il puntatore di un elemento e restituisce il puntatore incrementato

Le funzioni

- Le funzioni sono dei sottoprogrammi costituiti da un insieme di istruzioni
- Il loro uso permette di realizzare programmi più chiari da leggere, da capire e da modificare, infatti...
    - attraverso l'uso delle funzioni si evita di dover riscrivere più volte del codice
    - è possibile chiamare una funzione più volte in un programma

Le funzioni sono alla base della programmazione modulare

Creazione librerie (scanf, printf) che nascondono i dettagli implementativi

Funzioni: introduzione

`valori in ingresso` -> `FUNZIONE` -> `valore restituito`

Abbiamo già usato le funzioni!

```
int main() {

  return 0;
}
```

Funzioni: dichiarazione

- Scelta del nome della funzione
- Tipo del valore restituito
- Tipo (e nome) dei parametri in ingresso (parametri formali)

```
void nome_funzione(void);
```

oppure

```
int somma(int, int);
```

Funzioni: definizione

- Con la definizione di una funzione specifichiamo le istruzioni che caratterizzano la funzione

```
int somma(int c,int d) //nome dei parametri formali
{
  int s;
  s=c+d;
  return s; //restituiamo s al main (o alla funzione chiamante)
}
```

La definizione deve essere inserita subito dopo la chiusura del `main`

Funzioni: chiamata

- La chiamata permette di eseguire la funzione, passare i parametri (attuali) ed assegnare il valore restituito

```
int main() {
  int a=1,b=1,sum;

  sum=somma(a,b); //a e b sono i parametri attuali
  printf("\nLa somma vale %d\n",sum);
}
```

! Nella chiamata occorre fare attenzione che il tipo, il numero e l'ordine dei parametri attuali sia conforme ai parametri formali indicati nella definizione della funzione!

Void

```
#include <stdio.h>
void bmi(float,float);

int main() {
  float peso,altezza;

  printf("\nPeso in Kg: ");
  scanf("%f",&peso);
  printf("\nAltezza in m: ");
  scanf("%f",&altezza);
  bmi(peso,altezza);
}

void bmi(float p,float a) {

  printf("\nIl bmi vale %.2f\n",p/(a*a));
}
```

- Void viene utilizzata per eseguire un'operazione o un effetto collaterale, ma non restituisce alcun valore. Ad esempio, una funzione void potrebbe essere utilizzata per stampare un messaggio a schermo o per modificare una variabile globale

Funzioni: passaggio dei parametri

- In C avviene sempre e solo per valore
    - i parametri formali vengono inizializzati con i corrispondenti valori dei parametri attuali

Una funzione non modifica i valori dei parametri attuali

- Passaggio esplicito: utilizzo dei parametri attuali e formali
- Passaggio implicito: utilizzi di variabili globali (è buona norma evitarle!)

Visibilità (scope)

- La dichiarazione di una variabile introduce un nome simbolico visibile (scope) all'interno di una specifica parte (blocco) del programma
- Nome locale – variabile locale
    - visibilità solo all'interno del blocco delle istruzioni della funzione: dal punto della dichiarazione alla fine del blocco
- Nome globale – variabile globale
    - definita fuori dalla funzione: dal punto della dichiarazione alla fine del file I parametri formali sono delle variabili locali!

```c
#include <stdio.h>

int main() {

  for(int i=0;i<10;i++)
    printf("%d",i);
  printf("%d",i);

}
```

Cosa produce questo programma?

Stampa due valori uguali ma due volte. Es. i vale 5 allora stamperà 55

Visibilità (scope)

- Occorre prestare particolare attenzione al mascheramento
- Mascheramento: la dichiarazione di una variabile locale può nascondere (e quindi mascherare) una variabile con lo stesso nome dichiarata in un blocco più esterno (o globale)

```c
#include <stdio.h>

int quadrato(int);
int n=4; //variabile globale

int main() {
  int n=3; //ATTENZIONE al mascheramento

  printf("\nIl quadrato di %d vale %d\n",n,quadrato(n));
  //Il quadrato di 3 vale 9
}

int quadrato(int n) {
  n=n*n;
  return n;
}
```

- Parametri formali: definizione della funzione (tipo, numero e ordine)
- Parametri attuali: passati alla funzione quando viene chiamata

- In C il passaggio dei parametri avviene per valore, cioè durante la chiamata di una funzione ogni parametro formale viene inizializzato con il valore del corrispondente parametro attuale

- E se volessimo modificare il valore dei parametri attuali all'interno della funzione?
- Possiamo passare per valore l'indirizzo di una variabile cioè, inizializzare i parametri formali con l'indirizzo dei parametri attuali.

E quindi usare i puntatori!

```c
#include <stdio.h>
void scambia(int *, int *);

int main() {
  int x=0,y=1;
  printf("\nPrima x vale %d e y vale %d",x,y);
  scambia(&x, &y);
  printf("\nDopo x vale %d e y vale %d\n",x,y);
}

void scambia(int *x, int *y) {
  int tmp;
  tmp=*x;
  *x=*y;
  *y=tmp;
}
```

`Prima x vale 0 e y vale 1` e `Dopo x vale 1 e y vale 0`

Funzioni e Array

- E' possibile passare ad una funzione il valore di un elemento di un array? sì

```c
nome_funzione(nome_array[indice]);
```

- E' possibile passare ad una funzione un array? sì, ma occorre fare attenzione.

```c
nome_funzione(nome_array);
```

  Sto passando il puntatore al suo primo elemento!

- Perché i puntatori hanno un tipo?
- Perché i puntatori non vengono utilizzati solo per memorizzare l'indirizzo di una variabile!
  - Deferenziazione
  - Aritmetica dei puntatori

Stringhe

```
char carattere; //memorizza un solo carattere!
```

- Le stringhe sono array di caratteri (tipo char)

`char c[10];` <- vettore di 10 caratteri

- Non è indispensabile specificare il numero di elementi
- La dimensione sarà determinata da: inizializzazione + carattere null (\0)

```
char stringa[] = "Elementi di Informatica";
```

- Il carattere \0 ci permette di usare una stringa senza conoscerne l'effettiva dimensione!

```
#include <stdio.h>

int main() {
char stringa[]="Elementi di informatica";
int i=0;

while(stringa[i]!='\0') {
  printf("%c",stringa[i]);
  ++i;
}
}
```

Per visualizzare l'intera stringa si usa il formato `%s`

```
#include <stdio.h>

int main() {
char stringa[]="Elementi di informatica";

printf("%s",stringa);
}
```

Stringhe: copia

```c
#include <stdio.h>
#define DIM 100

int main()
{
    char stringa[]="Elementi di Informatica";
    char altra_stringa[DIM];
    int i;

    for(i=0;stringa[i]!='\0';i++)
      altra_stringa[i]=stringa[i];

    altra_stringa[i]='\0';
    printf("Prima: %s - Seconda: %s\n",stringa,
    altra_stringa);
}
```

Stringhe: concatena

```c
//concatena due stringhe
#include <stdio.h>
#define DIM 100

int main()
{
    char stringa1[]="Elementi di ";
    char stringa2[]="Informatica";
    char concat[DIM];
    int i,j;

    for(i=0;stringa1[i]!='\0';++i)
        concat[i]=stringa1[i];

    for(j=0;stringa2[j]!='\0';++j)
        concat[i+j]=stringa2[j];

        concat[i+j]='\0';


    printf("%s",concat);
}
```

Stringhe: input e scanf()

```c
#include <stdio.h>
#define DIM 10

int main()
{
    char stringa[DIM];

    printf("Inserisci una stringa: ");
    scanf("%s",stringa);

    printf("La tua stringa: %s",stringa);
}
```

- Non è necessario usare il simbolo `&`
- ATTENZIONE: lo spazio interrompe l'immissione

Stringhe: input e getchar()

```c
#include <stdio.h>
#define DIM 100

int main()
{
    char stringa[DIM];
    int i;

  printf("Inserisci testo: ");
  for(i=0;((stringa[i]=getchar())!='\n') && (i<DIM);i++);
  stringa[i]='\0';
  printf("Parola: %s",stringa);
}
```

Con `getchar()` riusciamo a leggere anche lo spazio

Attenzione al `;` alla fine del ciclo `for`

Stringhe: la libreria <string.h>

- Libreria per operare sulle stringhe:
  - strcpy(str_dest,str_orig)
  - strncpy(str_dest,str_orig,n_char)
  - strcat(str1, str2)
  - strlen(str) -> rende numero di caratteri
  - strcmp(str1, str2) -> rende 0 se uguali

Stringhe: conversione

- Libreria <string.h>
  - atoi(stringa) : converte una stringa in un int
  - atof(stringa) : converte una stringa in un double

Strutture

- Quando si usa una struttura?
  - Quando abbiamo necessità di trattare un insieme NON omogeneo di dati.

Esempio: memorizzare una data

```
int giorno, mese, anno;
```

- Tre variabili per ogni data
- Se serve un'altra data, dobbiamo dichiarare altre tre variabili
- Le tre variabili sono logicamente collegate
- Sarebbe utile poterle raggruppare! Usiamo le strutture

```
struct data
{
 int giorno;
 int mese;
 int anno;
};
```

Strutture: dichiarazione

```
struct data oggi;
struct data data_nascita;
```

Con la dichiarazione viene allocato lo spazio in memoria

Strutture: come accedere

```
nome_variabile.nome_membro
```

```
oggi.giorno=6;
oggi.mese=11;
oggi.anno=2018;
```

Esempio

```
#include <stdio.h>

 struct data {
 int giorno;
 int mese;
 int anno;
};

int main() {
struct data oggi;

printf("Inserisci la data di oggi: ");
scanf("%d%d%d",&oggi.giorno,&oggi.mese,&oggi.anno);
printf("Oggi: %d/%d/%d",oggi.giorno,oggi.mese,oggi.anno);
}
```

Puntatori e strutture

```
#include <stdio.h>
#define NOME 50
struct anagr
    {
        int matricola;
        char nome[NOME];
        char cognome[NOME];
    };

int main() {

    struct anagr studente;
    struct anagr *pointer;

    pointer=&studente;
```

```
    printf("\nNome studente: ");
    scanf("%s",studente.nome);
    printf("\nCognome studente: ");
    scanf("%s",studente.cognome);
    printf("\nMatricola: ");
    scanf("%d",&studente.matricola);

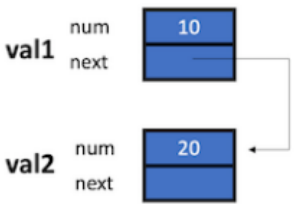
    printf("\n\nDati studente: ");
    printf("%s %s - matricola %d\n",(*pointer).nome,
    (*pointer).cognome,(*pointer).matricola);
}
```

Liste concatenate lineari

- Le liste lineari concatenate sono costituite da serie di elementi omogenei che occupano in memoria posizioni non adiacenti
- Puntatori a strutture e strutture contenenti puntatori sono di fondamentale importanza per la creazione e gestione delle liste concatenate lineari

Liste concatenate lineari (statiche)

```
#include <stdio.h>
struct lista_valori {
        int num;
        struct lista_valori *next;
    };
int main() {
    struct lista_valori val1,val2;
    val1.num=10;
    val2.num=20;
    val1.next=&val2;
    printf("%d",val1.next->num);
}
```



Liste concatenate lineari

- Alcune interessanti proprietà:
  - eliminare elementi
  - inserire elementi

Eliminare val2 dalla lista:

```
val1.next = val2.next;
```

Inserire nuovo elemento (val4) tra val2 e val3 :

```
val4.next = val2.next;
val2.next = &val4;
```

Questo è possibile grazie al fatto che gli elementi di una lista non sono memorizzati in sequenza.
Cosa succede negli array?

Gli array, a differenza delle liste, memorizzano gli elementi in posizioni di memoria contigue. Questo significa che ogni elemento in un array è immediatamente successivo all'elemento precedente nella memoria del computer. Questa struttura consente un accesso molto efficiente ai dati, poiché se conosci l'indirizzo di un elemento, puoi calcolare facilmente l'indirizzo di qualsiasi altro elemento.

Ad esempio, se hai un array di interi (dove ogni intero occupa 4 byte di memoria) e conosci l'indirizzo del primo elemento, puoi trovare l'indirizzo del terzo elemento semplicemente aggiungendo 8 (2 elementi * 4 byte/elemento) all'indirizzo del primo elemento

- Puntatore esterno al primo elemento della lista
  - `struct lista_valori *punt_lista = &val1;`
- Puntatore a NULL (fine lista)
  - `val3.next = NULL;`

Strutture e Array

- Strutture contenenti array
- Array di strutture
- Strutture contenenti strutture

Strutture contenenti Array

```c
#include <stdio.h>
#define NOME 20
    struct anagr
    {
        int matricola;
        char nome[NOME];
        char cognome[NOME];
    };
int main() {
    struct anagr studente;

    printf("\nNome studente: ");
    scanf("%s",studente.nome);
    printf("\nCognome studente: ");
    scanf("%s",studente.cognome);
    printf("\nMatricola: ");
    scanf("%d",&studente.matricola);

    printf("\n\nLo studente si chiama: ");
    printf("%s %s e la sua matricola e' %d\n",
    studente.nome,studente.cognome,studente.matricola);
}
```

Allocazione memoria

- Processo utilizzato per assegnare la memoria ad un programma
- Allocazione statica (in fase di compilazione - stack)
- Allocazione dinamica (in fase di esecuzione - heap)
- Lo stack è un'area della memoria (di dimensione fissa) che conserva le variabili locali, i parametri delle funzioni e le chiamate alle stesse funzioni
- Le variabili locali sono attive solo durante la chiamata ad una specifica funzione, dopo vengono "rilasciate"

Allocazione memoria: statica

- Stack:
  - Allocata / deallocata automaticamente dal compilatore
  - La dimensione è fissa e non può "crescere" (problema dello stack overflow)
  - Accesso più rapido
  - LIFO

Allocazione memoria: dinamica

- Heap:
  - Allocata / deallocata manualmente tramite l'uso di specifiche funzioni
  - La dimensione può cambiare ed è meno limitata (problema della frammentazione)
  - Nessun ordine specifico
  - Accesso meno rapido

Oggetti dinamici

- I puntatori possono essere usati per la creazione e la gestione di oggetti dinamici
- Gli oggetti dinamici sono creati durante l'esecuzione del programma
- Non è quindi necessaria una dichiarazione esplicita: non serve il nome e non serve conoscerne il numero a priori
- La memoria viene allocata attraverso l'utilizzo di specifiche funzioni: per esempio, malloc() (restituisce un puntatore)

Allocazione dinamica della memoria

- Utilizzo della funzione malloc() - serve stdlib.h (Standard LIary, libreria standard)
- Stdilib.h → Allocazione e deallocazione di memoria: calloc, malloc, realloc sono funzioni che si occupano dell'allocazione dinamica della memoria free libera la memoria allocata dinamicamente dalla famiglia di funzioni malloc ()
- La funzione malloc è una funzione della libreria standard per l'allocazione dinamica della memoria. Questa funzione è dichiarata nel file di intestazione stdlib.h
- Malloc è utilizzato per allocare un blocco di memoria nello heap del programma. Il programma accede a questo blocco di memoria tramite un puntatore che malloc restituisce. Quando la memoria non è più necessaria, può essere deallocata tramite la funzione free () in modo che possa essere utilizzata per altri scopi.

```
p = (int *) malloc (sizeof(int));
```

- Questa istruzione permette di allocare (dinamicamente) la memoria necessaria per un intero che sarà accessibile utilizzando il puntatore p
- (int *) è un cast al tipo specifico
    - Da questo momento possiamo accedere (tramite il puntatore p) a quella locazione di memoria e quindi all'intero senza l'utilizzo di una dichiarazione esplicita.


```
p = (int *) malloc (sizeof(int));
```


Possiamo modificare il valore puntato da p utilizzando la già nota rappresentazione:

```
*p=10;
```

E' necessario deallocare lo spazio per evitare che diventi successivamente inutilizzabile!

```
free(p);
```

Esempio: stack vs heap

```
#include <stdio.h>
#include <stdlib.h>

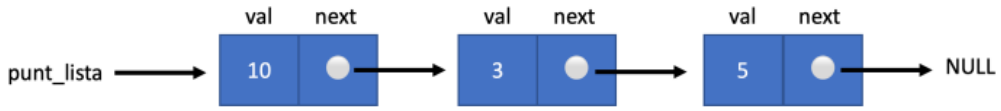
int main() {
    int *p; //p è nello stack

    p=(int *)malloc(sizeof(int));
    //malloc restituisce un indirizzo di heap
    *p=10; //modifica di heap

    printf("%p %p %d",&p,p,*p);
    }
```

Array e liste concatenate

- Array:
  - Occupazione memoria (sovrastima)
  - Velocità (inserimento/eliminazione e spostamento)
- Liste lineari:
  - Insieme di elementi omogenei memorizzati in una posizione qualsiasi (semplicità di inserimento/eliminazione e spostamento)



Array dinamico: malloc()

```c
#include <stdio.h>
#include <stdlib.h>

int main () {
    int i,n;
    int *ptr;

    printf("Inserisce dimensione array: ");
    scanf("%d",&n);

    ptr=(int*) malloc(n * sizeof(int));
     for(i=0;i<n;i++) {
         printf("\nInserisce valore: ");
         scanf("%d",ptr+i);
     }

    for(i=0;i<n;i++)
        printf("%d ",*(ptr+i));

    free(ptr);
}
```

Array dinamico: calloc()

```c
#include <stdio.h>
#include <stdlib.h>

int main () {
    int i,n;
    int *ptr;

    printf("Inserisce dimensione array: ");
    scanf("%d",&n);

    ptr=(int*) calloc(n, sizeof(int)); //init to 0!
     for(i=0;i<n;i++) {
         printf("\nInserisce valore: ");
         scanf("%d",ptr+i);
     }

    for(i=0;i<n;i++)
        printf("%d ",*(ptr+i));

    free(ptr);
}
```

Array dinamico: esempio di realloc()

```c
#include <stdio.h>
#include <stdlib.h>

int main() {
    int *p,*q, n=5, m,i;

    p=(int *) malloc(n * sizeof(int));
    for(i=0;i<n;i++) scanf("%d",p+i);
    printf("\n");
    for(i=0;i<n;i++) printf("%d ",*(p+i));
    m=10;
    q=realloc(p,m * sizeof(int));
    for(i=n;i<m;i++) scanf("%d",q+i);
    for(i=0;i<m;i++) printf("%d ",*(q+i));

    return 0;
}
```

I File

- Nel C è possibile operare su file solamente in termini di sequenze di byte: stream
- Non esistono funzioni di alto livello come in altri linguaggi
- Usiamo i puntatori. File di testo e file binari
    - Attenzione al SO
- Con i file si ha finalmente la possibilità di salvare i dati inseriti all'interno del programma

I File: apertura

- Prima di poter scrivere o leggere da file è necessario aprire il file stesso
- Per far questo è necessario includere la libreria stdio.h che contiene il tipo derivato FILE
    - Per leggere e scrivere su file si utilizza un puntatore (a tipo derivato FILE)

```
FILE *fileptr; //puntatore al file
```

I File: apertura e chiusura

- 1. fopen() associa un puntatore al file
- Oltre al nome del file occorre specificare il tipo di operazione da eseguire

```
FILE *fileptr;
fileptr = fopen("my_file", "r"); //r -> in lettura
```

- 2. fclose() chiude un file e permette di salvare i dati

```
fclose(fileptr);
```

I File: operazioni in apertura

- r : solo lettura – se il file non esiste fopen() restituisce errore (NULL)
- w : solo scrittura – se il file non esiste sarà creato, altrimenti sovrascritto
- r+ : lettura e scrittura – se il file non esiste fopen() restituisce errore (NULL)
- w+ : scrittura e lettura - se il file non esiste sarà creato, altrimenti sovrascritto
- a : solo scrittura a fine file (append) - se il file non esiste sarà creato, altrimenti mantenuto
- a+ : lettura e scrittura a fine file (append) - se il file non esiste sarà creato, altrimenti mantenuto

I File: funzioni per lettura e scrittura

- fprintf() e fscanf() : simili a printf() e scanf() ma interagiscono con i file e consentono di agire in modo "formattato"

I File: esempio `fprintf()`

```c
#include <stdio.h>

int main() {
  FILE *fileptr;
  int i;

  fileptr=fopen("file_01.txt","w");
  for(i=1;i<=5;i++)
    fprintf(fileptr,"%d ",i);
  fclose(fileptr);
}
```

I File: esempio `fscanf()`

```c
#include <stdio.h>

int main() {
  FILE *fileptr;
  int i,num;

  fileptr=fopen("file_01.txt","r");
  for(i=1;i<=5;i++) {
    fscanf(fileptr,"%d",&num);
    printf("%d ",num);
  }
  fclose(fileptr);
}
```

I File: fine file...

```c
#include <stdio.h>

int main() {
  FILE *fileptr;
  int num;

  fileptr=fopen("file_01.txt","r");
  while(fscanf(fileptr,"%d",&num)==1) printf("%d ",num);
  fclose(fileptr);
}
```

In C la funzione `feof()` rende un valore diverso da 0 quando arriva a fine file - ma attenzione

I File: altre funzioni per lettura e scrittura

- fgets() : legge una riga da un file
- fputs() : scrive una riga su un file
- Il fine riga è individuata dal carattere \n

I File: `fgets()`

```c
#include <stdio.h>
#define DIM 100
int main() {
    FILE *fileptr;
    char vettore[DIM];
    fileptr=fopen("file_01.txt","r");
    if(fileptr==NULL) printf("\nFile inesistente\n");
    else {
        fgets(vettore,DIM,fileptr);
        printf("%s",vettore);
        fclose(fileptr);
    }
}
```

I File: fgets()

- s=fgets(vettore,DIM,fileptr)
- s indirizzo di vettore vettore
- vettore che conterrà la riga letta
- DIM dimensione del vettore
- fileptr puntatore al file da legger
- La funzione fgets() aggiunge a fine linea il carattere \0 di fine stringa!

I File: `fgets()` e `stdin`

```c
#include <stdio.h>
#define DIM 100

int main() {
    FILE *fileptr;
    char vettore[DIM];
    fileptr=fopen("file_02.txt","w");
    printf("Inserisci del testo: ");
    fgets(vettore, DIM, stdin);
    fprintf(fileptr, "%s", vettore);
    fclose(fileptr);
    return 0;
}
```

I File: fgets() e stdin

- La funzione fgets() può quindi essere sfruttata per leggere una riga da tastiera (superando i limiti della scanf()) dirottando l'input dal file alla tastiera (stdin)

I File: fputs()

- La funzione fputs() scrive una riga su un file fputs(vettore,fileptr);

I File: `fputs()`

```c
#include <stdio.h>
#define DIM 100
int main() {
    FILE *fileptr;
    char vettore[DIM],invio;
    int n,i;
    fileptr=fopen("file_03.txt","w");
    printf("\nQuante linee vuoi scrivere: ");
    scanf("%d",&n);
    scanf("%c",&invio);
    for(i=1;i<=n;i++) {
        printf("\nInserisci linea %d: ", i);
        fgets(vettore,DIM,stdin);
        fputs(vettore,fileptr);
    }
    fclose(fileptr);
}
```

I File: leggere e scrivere un singolo carattere

- fgetc() e fputc()
  - fgetc(fileptr) : restituisce un intero. Costante simbolica EOF (-1)
  - fputc(c,fileptr) - Richiede in ingresso il carattere da scriver

I File: note

- fflush(fileptr) : scarica su disco tutte le scritture contenute nel buffer
- Messagio a video: fprintf(stdout, "Messaggio per l'utente");
- La funzione gets() non si deve usare!

I File: fread() - (low-level reading)

- fread() permette di leggere un file, traferire (stream) su un vettore e restituire il numero di elementi letti
- Se è stato letto tutto il contenuto del file fread() restituisce 0
- La lettura avviene in sequenza spostando il puntatore del numero di byte necessario

```c
#include <stdio.h>
#define DIM 100

int main() {
  FILE *fileptr;
  char vettore[DIM];
  int size=1; //in byte di 1 elemento
  int n; //num elementi letti
  int i;

  fileptr=fopen("file01","r"); //path!

  if(fileptr==NULL)
    printf("\nFile inesistente\n");
  else {
    n=fread(vettore,size,DIM,fileptr);
      for(i=0;i<n;i++)
        printf("%c",vettore[i]);
    fclose(fileptr);
  }
}
```

I File: fwrite() - (low-level writing)

- fwrite() permette di scrivere su un file, traferire (stream) da un vettore ciò che legge e restituire il numero di elementi scritti
- La scrittura avviene in sequenza spostando il puntatore del numero di byte necessario.

I File: esempio di scrittura

```c
#include <stdio.h>
#include <string.h>
#define DIM 100

int main() {

  FILE *fileptr;
  char vettore[DIM];
  int size=1,n,i,len;

  printf("Inserisci del testo: ");
  scanf("%s",vettore);
  len=strlen(vettore);

  fileptr=fopen("file02","w");
  n=fwrite(vettore,size,len,fileptr);
  fclose(fileptr);
}
```

```c
#include <stdio.h>
int main() {
    FILE *fileptr;
    int vettore[5],i;
    fileptr=fopen("file_04","wb");
    if(fileptr==NULL) printf("\nFile inesistente\n");
    else {
        for(i=0;i<5;i++)
            scanf("%d",&vettore[i]);
        fwrite(vettore,sizeof(int),5,fileptr);
        fclose(fileptr);
    }
    fileptr=fopen("file_04","rb");
    if(fileptr==NULL) printf("\nFile inesistente\n");
    else {
        fread(vettore,sizeof(int),5,fileptr);
        printf("\nElementi in array:");
        for(i=0;i<5;i++)
            printf("\n%d",vettore[i]);
        fclose(fileptr);
    }
}
```

Un'unica app per tutte le esigenze di Word, Excel, PowerPoint e PDF. Scarica l'app Microsoft 365: https://aka.ms/GetM365